Table des matières

Exercices corrigés 09 — Streaming et événements : MQTT et Kafka 1
Questions courtes 1
Questions pratiques 2
Diagnostics 3
Questions à développer 3
Mises en situation 4
Corrigé 5

Exercices corrigés 09 — Streaming et événements : MQTT et Kafka

Comment travailler ces exercices. Répondez seul, puis vérifiez avec le corrigé. Les exercices marqués [labo] portent sur les programmes que vous venez d'écrire et sur les chiffres que vous avez sous les yeux ; les autres vérifient que le cours est clair dans votre tête. Tout vient de la Théorie 09 et du Labo 09.

Les chiffres à retrouver. Les questions pratiques supposent le labo fini : le courtier et Kafka démarrés, les trois sujets remplis par le rejeu d'octobre et par la semaine de télémétrie, et le schéma `temps_reel` chargé. Les requêtes sur l'archive se tapent dans la ligne de commande DuckDB, lancée depuis `Projet_Boreal` ; celles sur la base, dans `psql`, comme à l'étape 14 du labo.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Les trois rôles. Nommez les trois rôles de MQTT, dites ce que fait chacun, et dites lequel garde les messages.
2. Les jokers. Dites ce que reçoit un abonné à `boreal/vehicules/+/sorties`, puis un abonné à `boreal/usine/#`. Dites enfin pourquoi `boreal/usine/#/VIBRATION` est refusé.
3. Trois qualités. Donnez, pour chacune des trois qualités de service, ce que l'abonné peut recevoir d'un message publié une fois.
4. Soixante-dix. La semaine de télémétrie rejouée porte 3 449 lignes pour 3 379 messages. D'où viennent les 70 lignes en trop, et à quoi les reconnaît-on ?
5. Le message retenu. Dites ce que le courtier garde d'un sujet où l'on publie des messages retenus, ce qu'un nouvel abonné en reçoit, et comment on efface ce qui est gardé.

© 2026 iAlexMG. Tous droits réservés.

6. La session persistante. Dites ce qu'elle garde pour un abonné absent, ce qui borne ce qu'elle garde chez Mosquitto, et ce qui arrive au-delà de la borne.
7. L'accusé de l'abonné. Le client paho acquitte un message de qualité 1 au retour de la fonction qui le traite. Dites pourquoi l'abonné du labo écrit chaque ligne sur le disque avant de rendre la main, et ce qui arriverait s'il écrivait plus tard.
8. Le journal. Définissez en une phrase chacun : un sujet Kafka, une partition, un décalage, la rétention. Donnez la rétention par défaut.
9. L'ordre. Dites ce que Kafka garantit sur l'ordre des messages, puis pourquoi le pont de Boréal prend l'équipement pour clé plutôt que l'étiquette.
10. Le groupe. Dites ce qu'un groupe de consommateurs neuf lit en premier avec `auto.offset.reset` à `earliest`, et combien de membres d'un même groupe peuvent lire utilement un sujet de trois partitions.
11. Deux heures. Distinguez le temps de l'événement et le temps de traitement, puis dites ce qu'est un filigrane.
12. Une seule fois, en effet. Donnez les deux règles qui font, chez Boréal, qu'un fait se retrouve une seule fois dans la base alors que chaque maillon livre au moins une fois.

Questions pratiques

1. [labo] Les doublons, jour par jour. Écrivez la requête qui compte, sur l'archive de la semaine du 10 au 16 janvier 2026, les doublons par jour de réception. Dites ce que vous attendez de la fin de semaine, et vérifiez.
2. [labo] Le plus en retard. Écrivez la requête qui donne, sur la même semaine, les trois messages reçus le plus longtemps après la fin de leur sortie, avec le numéro de série, la fin et la réception. Dites ce que le premier nous apprend sur le fichier où il a été rangé.

1. [labo] Les codes de la semaine. Comptez, dans `temps_reel.sorties`, les messages par code d'anomalie. Puis dites combien des messages portant `T320` viennent des motoneiges de la plage de l'incident, de `MN26001238` à `MN26001466`.
2. [labo] La pression du réseau d'air. Écrivez la commande `mosquitto_sub` qui n'écoute que la pression du collecteur d'air, puis rejouez la journée du 24 octobre 2025. Dites combien de messages vous attendez, et d'où vient ce chiffre.
3. [labo] Les alertes gardées. Lisez le sujet `usine.alertes` depuis son début, avec les clés, par l'outil `kafka-console-consumer.sh` du conteneur de Kafka. Dites combien de messages vous obtenez et pourquoi ce nombre.
4. [labo] Un seuil plus bas. Relisez `usine.points` par un groupe neuf, avec un seuil d'alerte de 3,0 mm/s et sans rien écrire. Donnez la date de la levée, et dites si un autre compresseur que le 2 déclenche quoi que ce soit.
5. [labo] Par la fin ou par la réception. Comptez les sorties de la semaine par jour de fin, puis par jour de réception. Expliquez les trois premières lignes, et dites lequel des deux comptes un tableau de bord des sorties quotidiennes doit afficher.
6. [labo] Simple ou pondérée. La fenêtre de 8 h à 8 h 15 le 22 octobre 2025 a levé l'alerte à 4,545 mm/s. Calculez dans `psql` la moyenne simple des points de vibration du compresseur 2 dans cette fenêtre, et comparez.

© 2026 iAlexMG. Tous droits réservés.

7. [labo] Trois décalages. Après la semaine de télémétrie, le groupe `vers-series` a lu les trois partitions de `vehicules.sorties` jusqu'aux décalages 1 142, 1 175 et 1 132. Dites ce que vaut leur somme, et pourquoi elle ne vaut pas 3 379.

8. [labo] Un seuil plus haut. Même question que la pratique 6, avec un seuil de 5,0 mm/s. Donnez la date de la levée, et l'avance qui reste sur le déclenchement.

Diagnostics

1. Un collègue compte les sorties de la semaine en comptant les lignes de l'archive, et annonce 3 449 sorties au responsable des garanties. Qu'est-ce qui est faux, et comment le corrigez-vous sans toucher à l'archive ?

2. Pour aller plus vite, un développeur change la qualité de service des résumés de sortie de 1 à 0. Ses doublons disparaissent. Que perd-il en échange, et quand s'en apercevra-t-il ?

3. Un abonné tardif écrit chaque message qu'il reçoit dans le fichier du jour, sans regarder s'il est retenu. Au lendemain d'un redémarrage, le tableau de bord de l'usine montre 58 points de plus que l'historien. Expliquez, puis corrigez.

4. Un pont appelle `producteur.produce()` puis rend la main, sans attendre la confirmation de Kafka et en laissant paho acquitter. Tout fonctionne pendant des semaines. Décrivez la panne qui lui fera perdre des messages, et le correctif.

5. Un consommateur enregistre son décalage dans Kafka, puis écrit le lot dans la base. Il plante une fois entre les deux. Que manque-t-il, et comment l'ordre inverse, avec la clé primaire, règle-t-il le problème ?

6. Une ingénieure clé les points de l'usine par étiquette. L'alerte lève parfois une fenêtre sur un compresseur arrêté, ou en saute une qu'elle aurait dû juger. Expliquez le mécanisme, et dites pourquoi le défaut ne se voit pas à chaque rejeu.

7. Un analyste calcule ses fenêtres de 15 minutes sur l'heure de réception, par l'horloge du poste, pendant un rejeu à 3 600 fois. Il obtient une douzaine de fenêtres pour tout octobre, dont les moyennes ne ressemblent à rien. Expliquez.

8. Une alarme de vibration est réglée pour se lever au-dessus de 4,5 mm/s et retomber dès qu'elle repasse dessous. Les opérateurs se plaignent qu'elle sonne sans cesse le premier jour d'une dérive, puis la désactivent. Expliquez, chiffrez avec Boréal, et corrigez.

Questions à développer

A. La direction de Boréal veut « tout passer en temps réel ». En vous appuyant sur l'alerte de vibration, sur ce qu'elle gagne par rapport à un calcul quotidien et sur ce que coûte le chemin chaud, écrivez la réponse que vous lui feriez, en disant où le temps réel vaut son prix et où il ne le vaut pas.

B. Comparez Lambda et Kappa sur le cas de la télémétrie de Boréal : ce que chacune ferait des 241 sorties arrivées après un filigrane d'une heure, ce qu'elle coûte à tenir, et celle que vous choisiriez pour un calcul de réclamations de garantie.

© 2026 iAlexMG. Tous droits réservés.

C. « Exactement une fois » est souvent vendu comme une propriété d'un outil. Montrez, à partir de la chaîne de Boréal, que c'est une propriété de toute la chaîne, et dites ce que la qualité de service 2 de MQTT et les transactions de Kafka apportent ou n'apportent pas.

Mises en situation

1. Une coopérative laitière fictive de Lotbinière équipe ses quarante fermes de capteurs sur les réservoirs de lait : température toutes les 30 secondes, volume à chaque traite. Elle veut un tableau de bord du volume par heure, et une alerte quand un réservoir dépasse 4 °C plus de dix minutes. On lui a conseillé MQTT, Kafka, un moteur de flux et une base de séries temporelles. Dites ce que vous garderiez, ce que vous retireriez, et pourquoi.

2. Une société de vélos partagés fictive de Beauport reçoit un événement à chaque départ et à chaque retour de vélo. Une borne privée de réseau garde ses événements et les envoie quand elle revient, parfois six heures plus tard. La direction veut le nombre de trajets par quart d'heure, en direct et pour le rapport du mois. Proposez le calcul, le traitement des retards et ce que vous affichez en direct.

3. Un parc éolien fictif du Témiscouata publie chaque seconde une vingtaine de mesures pour chacune de ses quarante éoliennes. L'exploitation veut des moyennes de dix minutes et des alarmes ; l'équipe de science des données veut pouvoir rejouer deux ans de données pour entraîner un modèle. Proposez les sujets, les clés, les partitions et la rétention, et dites où vivent les deux ans.

4. (avancé) Une scierie fictive de Mont-Laurier doit relier son courtier MQTT à Kafka. Un intégrateur propose Kafka Connect avec un connecteur MQTT commercial ; un développeur de l'équipe propose d'écrire le pont lui-même, comme au labo. Donnez les critères de choix, ce que chaque option coûte sur trois ans, et ce que vous exigeriez de l'une comme de l'autre avant de la mettre en service.

© 2026 iAlexMG. Tous droits réservés.

Corrigé

Les chiffres cités sont ceux de la génération du cours, jusqu'à 2026-03, et des rejeux du labo : l'usine du 13 au 31 octobre 2025, la télémétrie du 10 au 16 janvier 2026. Les heures de réception d'un rejeu varient de quelques secondes d'un passage à l'autre ; les comptes tirés de l'archive, eux, ne varient pas.

Questions courtes

1. Les trois rôles. L'éditeur publie un message sur un sujet sans savoir qui l'écoute ; l'abonné déclare les sujets qui l'intéressent et reçoit ce qui s'y publie ; le courtier reçoit chaque message et le relaie à chaque abonné du sujet. Aucun des trois ne garde les messages : le courtier relaie, et ne conserve que ce qu'on lui demande, le dernier message retenu de chaque sujet et la file d'un abonné en session persistante.

Théorie : `Theorie_09_DE.md`, sections 2 et 5.

2. Les jokers. `boreal/vehicules/+/sorties` reçoit les résumés de sortie de tous les véhicules : `+` remplace exactement un niveau, le numéro de série. `boreal/usine/#` reçoit tous les points de l'usine, quelle que soit la profondeur : `#` remplace tous les niveaux qui suivent. `boreal/usine/#/VIBRATION` est refusé parce que `#` ne peut venir qu'en dernière position : il dit « tout le reste », et rien ne peut le suivre. Pour les vibrations de toute l'usine, on écrit `boreal/usine/+/+/VIBRATION`.

Théorie : `Theorie_09_DE.md`, section 3.

3. Trois qualités. En qualité 0, le message ou rien : au plus une fois. En qualité 1, le message, ou le message deux fois quand un accusé s'est perdu : au moins une fois. En qualité 2, le message exactement une fois entre deux clients MQTT, au prix de deux allers-retours ; ce qui arrive ensuite chez l'abonné échappe au protocole.

Théorie : `Theorie_09_DE.md`, section 4.

4. Soixante-dix. Ce sont les doublons de la qualité de service 1 : le module a publié, l'accusé s'est perdu, le message a été renvoyé, et l'abonné de Boréal l'a reçu deux fois. On les reconnaît à leur `message_id`, le même dans les deux exemplaires. Le rejoueur les republie tels quels, parce qu'il rejoue ce que l'abonné a reçu.

Théorie : `Theorie_09_DE.md`, section 4.

5. Le message retenu. Le courtier garde le dernier message retenu de chaque sujet, un seul. Un nouvel abonné le reçoit aussitôt, marqué comme retenu : chez Boréal, un abonné tardif reçoit d'un coup la dernière valeur des 58 étiquettes. On efface le message gardé en publiant sur le même sujet un message retenu vide, `mosquitto_pub -r -n`.

Théorie : `Theorie_09_DE.md`, section 5.

© 2026 iAlexMG. Tous droits réservés.

6. La session persistante. Elle garde les abonnements de l'abonné et, pendant son absence, ses messages de qualité 1 et 2, qu'elle lui livre à son retour. Chez Mosquitto, la file est bornée par `max_queued_messages`, 1 000 par défaut ; au-delà, les messages sont jetés sans avertissement. Mesuré au labo avec ce réglage, un abonné absent pendant que 2 118 messages passaient n'en a retrouvé que 999. La plateforme relève la borne à 100 000.

Théorie : `Theorie_09_DE.md`, section 5.

7. L'accusé de l'abonné. Parce que l'accusé part dès que la fonction rend la main : ce qui n'est pas encore sur le disque à ce moment est acquitté sans être en lieu sûr. Si l'abonné gardait ses lignes dans un tampon et mourait avant de le vider, le courtier, qui a reçu l'accusé, ne les relivrerait jamais, et l'archive aurait un trou que rien ne signale. `buffering=1` fait partir chaque ligne avant l'accusé.

Théorie : `Theorie_09_DE.md`, section 4.

8. Le journal. Un sujet est un journal nommé, où l'on ne fait qu'ajouter au bout. Une partition est un morceau de ce journal, avec son propre ordre et ses propres positions. Un décalage est la position d'un message dans sa partition, qui ne change jamais. La rétention dit combien de temps ou d'octets le journal garde ; par défaut, sept jours (`log.retention.hours` vaut 168) et aucune borne de taille.

Théorie : `Theorie_09_DE.md`, section 7.

9. L'ordre. Kafka ne garantit l'ordre qu'à l'intérieur d'une partition, et une même clé va toujours dans la même partition. L'alerte juge la vibration pendant la marche : elle a besoin de l'état de marche et de la vibration du même compresseur, dans l'ordre où ils se sont produits. Avec l'étiquette pour clé, ces deux points pourraient tomber dans deux partitions et se lire dans le désordre ; avec l'équipement pour clé, les quatre points du compresseur 2 partagent une partition.

Théorie : `Theorie_09_DE.md`, section 8.

10. Le groupe. Il lit en premier le plus ancien message que la rétention garde, puis tout le reste, à son rythme. Trois membres au plus lisent utilement un sujet de trois partitions, une partition n'étant lue que par un seul membre à la fois ; un quatrième attend sans rien faire.

Théorie : `Theorie_09_DE.md`, section 9.

11. Deux heures. Le temps de l'événement est celui où la chose s'est produite, l'instant d'un point ou la fin d'une sortie ; le temps de traitement est celui où le programme la voit. Un filigrane est l'affirmation qu'on ne recevra plus rien d'antérieur à tel instant : une fenêtre au temps de l'événement se ferme quand le filigrane dépasse sa fin, et ce qui arrive ensuite est en retard.

Théorie : `Theorie_09_DE.md`, section 11.

12. Une seule fois, en effet. L'écriture est idempotente : chaque table a une clé primaire qui identifie le fait, et chaque écriture passe par `ON CONFLICT DO NOTHING`. Et la base se valide avant le décalage : un lot est d'abord écrit, puis seulement sa position enregistrée dans Kafka, si bien qu'une panne entre les deux fait relire un lot que la clé écarte.

Théorie : `Theorie_09_DE.md`, section 13.

© 2026 iAlexMG. Tous droits réservés.

Questions pratiques

1. [labo] Les doublons, jour par jour. Dans la ligne de commande DuckDB :

```
SELECT CAST(CAST(recu_le AS TIMESTAMPTZ) AS DATE) AS jour,
       count(*) - count(DISTINCT message->>'message_id') AS doublons
FROM read_json('sources/telemetrie/sorties_2026-01-1[0-6].jsonl',
     columns = {sujet: 'VARCHAR', recu_le: 'VARCHAR', qos: 'INTEGER', message: 'JSON'})
GROUP BY jour
ORDER BY jour;
```

   La session doit être à l'heure de l'Est, `SET TimeZone = 'America/Toronto';`, pour que le jour soit celui du fichier. On attend plus de doublons la fin de semaine, puisqu'un doublon est une fraction à peu près constante des messages, et que la fin de semaine en porte cinq fois plus : 25 le samedi 10, 20 le dimanche 11, puis 3, 9, 2, 3 et 8. Le total fait 70.

Théorie : `Theorie_09_DE.md`, section 4.

2. [labo] Le plus en retard. On garde le premier exemplaire de chaque message, puis on trie par l'écart :

```
WITH p AS (
  SELECT message->>'message_id' AS id, any_value(message->>'sernr') AS sernr,
         CAST(any_value(message->>'fin') AS TIMESTAMPTZ) AS fin,
         min(CAST(recu_le AS TIMESTAMPTZ)) AS recu
  FROM read_json('sources/telemetrie/sorties_2026-01-1[0-6].jsonl',
       columns = {sujet: 'VARCHAR', recu_le: 'VARCHAR', qos: 'INTEGER', message: 'JSON'})
  GROUP BY id)
SELECT sernr, fin, recu, recu - fin AS retard FROM p ORDER BY retard DESC LIMIT 3;
```

   Le premier est la motoneige `MN25001293` : sortie finie le 9 janvier à 21 h 06, message reçu le 13 à 21 h 06 min 24 s, quatre jours plus tard. Suivent `HR25000944`, 2 jours 18 h, et `MN26000788`, 2 jours 10 h. Le premier est rangé dans le fichier du 13 janvier, jour de sa réception : qui charge ce fichier trouve une sortie du 9, et qui a déjà calculé le 9 l'a calculé sans elle.

Théorie : `Theorie_09_DE.md`, section 11.

3. [labo] Les codes de la semaine. Dans `psql` :

```
SELECT code, count(*) AS messages
FROM temps_reel.sorties, unnest(codes) AS code
GROUP BY code
ORDER BY code;
```

   `D240` 11, `H510` 4, `T310` 2, `T320` 8. Pour la plage de l'incident :

© 2026 iAlexMG. Tous droits réservés.

```
SET TIME ZONE 'America/Toronto';
SELECT sernr, fin FROM temps_reel.sorties
WHERE 'T320' = ANY (codes) AND sernr BETWEEN 'MN26001238' AND 'MN26001466'
ORDER BY fin;
```

Trois des huit messages `T320`, venus de deux motoneiges : `MN26001457` le 10 janvier, `MN26001364` le 13 et le 16. Le niveau de liquide bas, sur des colliers serrés sous le couple minimal en octobre. La table sert au Cours 11 et à Analytics Engineering, qui remonteront de ces codes à la plage.

Théorie : `Theorie_09_DE.md`, section 6.

4. [labo] La pression du réseau d'air. Dans Ubuntu, depuis `plateforme` :

```
docker compose exec mqtt mosquitto_sub -t 'boreal/usine/UTL/AIR/PRESSION' -v
```

Puis, depuis `Data_Domain`, le rejeu du 24 octobre. On attend 265 messages : c'est le nombre de points de la pression ce jour-là au Labo 08, valeurs de départ exclues, et le rejoueur publie exactement ces points. C'est aussi le pic de la dérive, contre 89 points par jour en marche normale.

Théorie : `Theorie_09_DE.md`, section 3.

5. [labo] Les alertes gardées. Dans Ubuntu, depuis `plateforme` :

```
docker compose exec kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic usine.alertes --from-beginning --property print.key=true --timeout-ms 5000
```

L'outil affiche quatre lignes, la clé puis la valeur, séparées par une tabulation ; les voici abrégées :

```
UTL-CMP2 {"actif": "UTL-CMP2", "niveau": "ALERTE", "etat": "levée", "fenetre_debut": "2025-10-22T08:00:00-04:00", ...}
UTL-CMP2 {"actif": "UTL-CMP2", "niveau": "ARRET", "etat": "levée", "fenetre_debut": "2025-10-27T06:00:00-04:00", ...}
UTL-CMP2 {"actif": "UTL-CMP2", "niveau": "ALERTE", "etat": "retombée", "fenetre_debut": "2025-10-30T13:45:00-04:00", ...}
UTL-CMP2 {"actif": "UTL-CMP2", "niveau": "ARRET", "etat": "retombée", "fenetre_debut": "2025-10-30T13:45:00-04:00", ...}
Processed a total of 4 messages
```

Quatre messages, un par changement d'état : deux levées et deux retombées. La relecture de l'étape 15 et les groupes des pratiques 6 et 10 n'y ont rien ajouté, grâce à `--essai`. Le message `ERROR Error processing message, terminating consumer process` suivi d'une `TimeoutException`, qui s'affiche après cinq secondes sans rien de neuf, n'est pas une panne : c'est la façon dont `--timeout-ms` met fin à la lecture.

Théorie : `Theorie_09_DE.md`, section 7.

6. [labo] Un seuil plus bas. Depuis `01_de` :

```
python -m flux.alerte --groupe seuil-trois --seuil 3.0 --essai --silence 5
```

```
Groupe seuil-trois, alerte à 3 mm/s, arrêt à 7.1 mm/s.
2025-10-20 06:15  UTL-CMP2  ALERTE levée : 3.065 mm/s sur la fenêtre de 06:00
2025-10-27 06:15  UTL-CMP2  ARRET levée : 7.554 mm/s sur la fenêtre de 06:00
```


© 2026 iAlexMG. Tous droits réservés.

```
2025-10-30 14:00  UTL-CMP2  ARRET retombée : 2.503 mm/s sur la fenêtre de 13:45
2025-10-30 14:15  UTL-CMP2  ALERTE retombée : 2.453 mm/s sur la fenêtre de 14:00
```

L'alerte se lève le 20 octobre à 6 h 15, deux jours plus tôt qu'au seuil de la procédure, et aucun autre compresseur ne déclenche quoi que ce soit : les compresseurs 1 et 3 restent sous 3,0 mm/s sur toute la période rejouée. Un détail de la dernière ligne : l'alerte retombe un quart d'heure après l'arrêt, parce que 2,503 n'est pas sous 3,0 moins la bande morte, 2,5. Un seuil plus bas donne plus d'avance, et c'est ce qu'on paie en fausses alertes sur d'autres périodes : la question se tranche sur toute l'année, pas sur un seul incident.

Théorie : `Theorie_09_DE.md`, sections 9 et 12.

7. [labo] Par la fin ou par la réception. Sur l'archive, le premier exemplaire de chaque message :

```sql
WITH p AS (
  SELECT message->>'message_id' AS id,
         CAST(any_value(message->>'fin') AS TIMESTAMPTZ) AS fin,
         min(CAST(recu_le AS TIMESTAMPTZ)) AS recu
  FROM read_json('sources/telemetrie/sorties_2026-01-1[0-6].jsonl',
       columns = {sujet: 'VARCHAR', recu_le: 'VARCHAR', qos: 'INTEGER', message: 'JSON'})
  GROUP BY id)
SELECT jour, sum(par_fin) AS par_fin, sum(par_reception) AS par_reception
FROM (SELECT CAST(fin AS DATE) AS jour, 1 AS par_fin, 0 AS par_reception FROM p
      UNION ALL
      SELECT CAST(recu AS DATE), 0, 1 FROM p)
GROUP BY jour
ORDER BY jour;
```

Les trois premières lignes, le 7, le 8 et le 9 janvier, portent 1, 1 et 16 sorties par leur fin et aucune par leur réception : ce sont les 18 sorties terminées avant la semaine, reçues pendant. Ensuite, les deux comptes ne diffèrent que de quelques unités par jour, 1 068 contre 1 069 le 10, 1 010 contre 1 004 le 11. Un tableau des sorties quotidiennes affiche le compte par la fin : une sortie appartient au jour où elle a eu lieu. Mais il doit dire qu'un jour récent est encore incomplet, puisque ses retardataires peuvent arriver pendant quatre jours.

Théorie : `Theorie_09_DE.md`, section 11.

8. [labo] Simple ou pondérée. Dans `psql` :

```sql
SELECT count(*) AS points, round(avg(valeur)::numeric, 3) AS moyenne_simple
FROM temps_reel.points
WHERE etiquette = 'UTL.CMP2.VIBRATION'
  AND instant >= TIMESTAMPTZ '2025-10-22 08:00:00-04'
  AND instant <  TIMESTAMPTZ '2025-10-22 08:15:00-04';
```


© 2026 iAlexMG. Tous droits réservés.

```
 points | moyenne_simple
--------+----------------
      2 |          4.565
```

Deux points seulement tombent dans la fenêtre : 4,64 à 8 h 08 et 4,49 à 8 h 14. Leur moyenne simple vaut 4,565. La moyenne pondérée vaut 4,545 parce qu'elle compte aussi la valeur en vigueur quand la fenêtre s'ouvre, 4,48, publiée à 7 h 56 et restée valable jusqu'à 8 h 08, huit minutes sur quinze. La moyenne simple ignore la valeur qui a duré le plus longtemps : c'est le défaut du Cours 08, au grain du quart d'heure. Ici, les deux dépassent 4,5 ; sur une autre fenêtre, l'une lèverait l'alerte et l'autre non.

Théorie : `Theorie_09_DE.md`, section 12.

9. [labo] Trois décalages. La somme vaut 3 449 : un décalage compte les messages écrits dans la partition, et le pont a écrit dans Kafka chaque ligne reçue du courtier, doublons compris. Le dédoublonnage se fait dans la base, par la clé primaire de `temps_reel.sorties`, qui n'en garde que 3 379. Kafka garde tout ce qu'on lui confie, et ce n'est pas à lui de trier.

Théorie : `Theorie_09_DE.md`, sections 7 et 13.

10. [labo] Un seuil plus haut. Depuis `01_de` :

```
python -m flux.alerte --groupe seuil-cinq --seuil 5.0 --essai --silence 5
```

```
Groupe seuil-cinq, alerte à 5 mm/s, arrêt à 7.1 mm/s.
2025-10-23 06:45  UTL-CMP2  ALERTE levée : 5.063 mm/s sur la fenêtre de 06:30
```

L'alerte se lève le 23 octobre à 6 h 45, un jour plus tard qu'au seuil de 4,5. Il reste 5 jours 3 h 59 min avant le déclenchement du 28 à 10 h 44 : encore bien assez pour planifier la réparation, mais plus assez pour que l'alerte coïncide avec le bruit signalé le 22.

Théorie : `Theorie_09_DE.md`, sections 1 et 9.

Diagnostics

1. Il a compté des lignes, pas des sorties : 70 des 3 449 lignes sont des doublons de la qualité de service 1, reçus deux fois avec le même `message_id`. Il y a 3 379 sorties. On corrige à la lecture, sans toucher à l'archive, qui doit rester ce que l'abonné a reçu : on compte les identifiants distincts, `count(DISTINCT message->>'message_id')`, ou on garde le premier exemplaire de chaque identifiant. Dans la base, la clé primaire le fait d'elle-même.

Théorie : `Theorie_09_DE.md`, sections 4 et 6.

2. Il perd la garantie de livraison. En qualité 0, un message dont le paquet se perd n'est jamais renvoyé, et rien ne le signale ni au module ni à l'abonné. Les doublons disparaissent parce que plus rien n'est renvoyé, les pertes comprises. Il s'en apercevra tard, et par hasard : une réclamation de garantie sans la sortie qui l'explique, ou un écart entre le compteur d'heures du moteur et la somme des sorties. Le bon remède était inverse : garder la qualité 1, et dédoublonner en aval par l'identifiant.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_09_DE.md`, section 4.

3. Au redémarrage, l'abonné s'abonne de nouveau, et le courtier lui livre aussitôt le dernier message retenu de chacune des 58 étiquettes. Ce ne sont pas des mesures neuves, mais des valeurs déjà reçues la veille, qu'il écrit une seconde fois. Le correctif : tester le drapeau `retain` du message reçu, et écarter en le comptant tout message retenu, comme le fait `Recepteur` au labo.

Théorie : `Theorie_09_DE.md`, section 5.

4. La panne : le pont meurt, ou Kafka tombe, entre l'accusé de paho et l'écriture par Kafka. Le message est alors acquitté auprès du courtier, qui l'oublie, et il n'est pas encore écrit : il est perdu, et rien ne le dira. Plus le producteur groupe ses envois, plus la fenêtre est large. Le correctif est celui du labo : se déclarer en accusé manuel, et n'acquitter le message que dans le rappel de livraison du producteur, quand Kafka a confirmé.

Théorie : `Theorie_09_DE.md`, section 10.

5. Il manque le lot entier. Kafka croit le lot écrit, puisque le décalage est enregistré ; la base ne l'a pas. Au redémarrage, le consommateur reprend après le lot, et ne le relira jamais. Dans l'ordre inverse, le lot est écrit, puis le décalage enregistré. Une panne entre les deux fait relire le lot, qui tente d'entrer une seconde fois : la clé primaire et `ON CONFLICT DO NOTHING` l'écartent sans erreur. On passe d'« au plus une fois » à « exactement une fois, en effet ».

Théorie : `Theorie_09_DE.md`, section 13.

6. Avec l'étiquette pour clé, `UTL.CMP2.MARCHE` et `UTL.CMP2.VIBRATION` peuvent tomber dans deux partitions différentes, et Kafka ne garantit aucun ordre entre partitions. Le consommateur de l'alerte peut alors lire une vibration avant l'état de marche qui la précédait : il la compte comme mesurée à l'arrêt, ou l'inverse, et la fenêtre est jugée sur un mauvais état. Le défaut dépend du rythme auquel le consommateur tire sur chaque partition, donc du hasard de l'exécution : un rejeu passe, le suivant non. Le correctif est la clé par équipement, qui met les quatre points d'un compresseur dans la même partition.

Théorie : `Theorie_09_DE.md`, section 8.

7. À 3 600 fois, quinze minutes du poste font quinze jours de Boréal : chaque fenêtre au temps de traitement mélange deux semaines de points, arrêts, nuits et dérive compris, et les moyennes ne décrivent plus rien. Une fenêtre sur la vibration se calcule au temps de l'événement, l'instant du point, et se ferme quand arrive un point plus récent que sa fin. Le défaut serait moins visible en production, où les deux horloges avancent au même pas, mais il y resterait : un message en retard y tomberait dans la mauvaise fenêtre.

Théorie : `Theorie_09_DE.md`, section 11.

8. Le premier jour d'une dérive, la moyenne oscille autour du seuil, et une alarme sans bande morte se lève à chaque passage au-dessus et retombe à chaque passage dessous. Chez Boréal, le 22 octobre 2025, elle se serait levée six fois, puis une septième le 23 au matin. Une alarme qui clignote cesse d'être lue, et les opérateurs ont fini par la couper, ce qui est pire que pas d'alarme du tout. Le correctif est une bande morte : se lever à 4,5 mm/s, ne retomber que sous 4,0. L'alarme se lève alors une seule fois, le 22 à 8 h 15, et retombe le 30 après la réparation.

Théorie : `Theorie_09_DE.md`, section 12.

© 2026 iAlexMG. Tous droits réservés.

Questions à développer

A. La réponse commence par une mesure. L'alerte de vibration en continu se lève le 22 octobre à 8 h 15 ; le même calcul, lancé chaque nuit sur la veille, l'aurait levée le 23 peu après minuit. Le temps réel gagne une quinzaine d'heures sur une avance de six jours. Pour le déclenchement du 28, les deux auraient suffi.

Ce que ces quinze heures valent vraiment se lit dans la GMAO : le 22 à 9 h 12, un technicien écoute le compresseur 2 pour un bruit signalé, et ne trouve rien. Avec l'alerte, levée depuis presque une heure, il savait quoi chercher. Le temps réel vaut donc son prix quand une décision se prend pendant l'événement : arrêter une machine, prévenir une équipe, retenir un lot avant qu'il parte. Il ne le vaut pas quand le résultat sert un rapport, une facture, un modèle entraîné chaque semaine, ou un tableau de bord qu'on regarde le lendemain.

Le coût, enfin, est permanent : deux services de plus sur la plateforme, des programmes à état qu'on surveille jour et nuit, et des erreurs nouvelles, doublons, retards, désordre, qu'il faut gérer à chaque maillon. La recommandation : un chemin chaud étroit, pour les quelques décisions qui se prennent sur le moment, et le chemin froid pour tout le reste, qui recalcule la vérité chaque nuit.

Théorie : `Theorie_09_DE.md`, sections 1 et 14.

B. Avec Lambda, le chemin chaud ferme ses fenêtres une heure après leur fin et écarte les 241 sorties arrivées plus tard ; le lendemain, le chemin froid recharge l'archive du jour, où elles sont rangées à leur jour de réception, et recalcule les jours passés avec elles. Le résultat du matin est approché, celui du lendemain est juste. Le coût : deux logiques qui doivent rendre le même chiffre, et un écart à surveiller entre elles.

Avec Kappa, on ne garde que le flux. Soit on garde les fenêtres ouvertes quatre jours, le plus grand retard de la semaine, ce qui tient en mémoire des milliers de fenêtres par véhicule ; soit on relit périodiquement le journal depuis le début avec la même version du programme, ce qui suppose une rétention bien plus longue que les sept jours par défaut. Le coût est dans la mémoire, ou dans le disque.

Pour un calcul de réclamations de garantie, Lambda : le résultat sert des décisions de paiement et des rapports, qui peuvent attendre la nuit et doivent être justes, et le chemin froid de Boréal existe déjà. Le chemin chaud reste pour ce qui est urgent, un code de surchauffe sur une motoneige de la plage, par exemple.

Théorie : `Theorie_09_DE.md`, sections 11 et 14.

C. Chez Boréal, aucun maillon ne livre exactement une fois : le courtier en qualité 1 livre au moins une fois, le pont n'acquitte qu'après Kafka et peut donc écrire un message deux fois s'il meurt entre l'écriture et l'accusé, et le consommateur relit un lot après une panne. Et pourtant chaque fait se retrouve une fois dans la base : 3 449 lus, 3 379 insérés, 70 écartés. L'unicité n'est dans aucun outil ; elle vient de la cible, qui sait reconnaître un fait déjà écrit, et de l'ordre des validations. C'est une propriété de la chaîne.

La qualité 2 de MQTT garantit l'unicité entre deux clients MQTT, pas au-delà : si l'abonné plante après avoir écrit et avant d'acquitter, il reçoit de nouveau. Les transactions de Kafka rendent atomiques plusieurs écritures dans Kafka et l'enregistrement d'un décalage : elles donnent l'unicité de bout en bout quand la source et la cible sont toutes deux Kafka. Pour une base extérieure, elles ne suffisent pas, et c'est l'écriture idempotente qui fait le travail.

Théorie : `Theorie_09_DE.md`, sections 4 et 13.

Mises en situation

© 2026 iAlexMG. Tous droits réservés.

1. Coopérative laitière de Lotbinière. On garde MQTT : c'est le protocole des capteurs, et un courtier léger suffit pour quarante fermes. On garde une base de séries temporelles, pour le volume par heure et l'historique des températures. On retire Kafka et le moteur de flux. Le débit est modeste, quarante réservoirs à deux mesures par minute, et personne ne demande de relire le flux ; un abonné qui écrit dans la base suffit, en qualité 1 et avec une clé qui dédoublonne. L'alerte des dix minutes au-dessus de 4 °C se calcule sur le temps des mesures, avec l'état gardé par réservoir, et une bande morte à la retombée. On ajoute ce que le courtier ne fait pas tout seul : une session persistante pour l'abonné, avec une file dimensionnée pour la plus longue coupure qu'on veut couvrir, et un contrôle qui signale un réservoir muet.

Théorie : `Theorie_09_DE.md`, sections 1, 5 et 12.

2. Vélos partagés de Beauport. Le trajet se compte au temps de l'événement, le départ, jamais à la réception. En direct, on calcule les quarts d'heure avec un filigrane court, une quinzaine de minutes, et l'on affiche le chiffre comme provisoire tant qu'une borne est muette : le tableau de bord dit « 3 bornes sans nouvelles ». Les événements arrivés après le filigrane ne sont pas jetés : ils sont comptés à part, et le chemin froid, qui recalcule chaque nuit à partir de l'archive complète, les intègre. Le rapport du mois se tire du chemin froid, et seulement après un délai qui dépasse le plus long retard observé, six heures ici. C'est Lambda, et le coût est de s'assurer que les deux chemins comptent de la même façon.

Théorie : `Theorie_09_DE.md`, sections 11 et 14.

3. Parc éolien du Témiscouata. Un sujet par nature de donnée, les mesures d'un côté, les alarmes de l'autre, et pour clé l'éolienne : ses mesures restent dans l'ordre, et une moyenne ou une alarme par machine se calcule sur une seule partition. Quarante clés se répartissent bien sur une douzaine de partitions, qui laissent de la marge pour un groupe de consommateurs plus large. La rétention se choisit selon la relecture qu'on veut permettre depuis Kafka : quelques semaines, pour rejouer un traitement corrigé. Les deux ans ne vivent pas dans Kafka : 800 mesures par seconde pendant deux ans font des dizaines de milliards de messages, qui se rangent dans un lac, en Parquet partitionné par jour, écrit par un consommateur. L'équipe de science des données rejoue depuis le lac ; Kafka rejoue ce qui est récent.

Théorie : `Theorie_09_DE.md`, sections 7, 8 et 14.

4. (avancé) Scierie de Mont-Laurier. Les critères : qui maintiendra le pont dans trois ans, ce qu'il doit garantir, et ce que l'équipe sait déjà faire tourner. Kafka Connect est fait pour ce travail : configuration plutôt que code, reprise sur panne, supervision uniforme avec les autres connecteurs. Son coût est une licence, si le connecteur est commercial, comme celui de Confluent d'après sa documentation, plus un service Connect à exploiter. Le pont maison ne coûte rien à l'achat, mais c'est du code à maintenir, à tester et à surveiller, et son auteur finira par partir. On exigerait de l'une comme de l'autre les mêmes preuves avant la mise en service : que l'accusé MQTT ne parte qu'après la confirmation de Kafka, mesurée en tuant le pont en plein débit ; que la clé choisie garde l'ordre dont les consommateurs ont besoin ; que le débit tienne la pointe, mesuré, avec la latence ; et qu'une coupure de Kafka ne perde rien, les messages restant chez le courtier. Pour une scierie sans développeur dédié, Connect ; pour une équipe qui a déjà écrit et testé le sien, le pont maison, documenté.

Théorie : `Theorie_09_DE.md`, sections 10 et 13.

© 2026 iAlexMG. Tous droits réservés.